

在 Cloud Run 部署、管理及觀測 ADK 代理

來源：<https://codelabs.developers.google.com/deploy-manage-observe-adk-cloud-run?hl=zh-tw>

步驟數：13

在本程式碼研究室中，您將在 Cloud Run 上部署 ADK 代理程式、管理新修訂版本的流量、觀察負載測試期間的代理程式服務行為，以及追蹤使用者執行的每個叫用程序。

目錄

1. 簡介
2. 🚀 準備研討會設定
3. 🚀 啟用 API
4. 🚀 Python 環境設定和環境變數
5. 🚀 準備 Cloud SQL 資料庫
6. 🚀 使用 ADK 和 Gemini 2.5 建構天氣代理
7. 🚀 部署至 Cloud Run
8. 💡 Dockerfile 和後端伺服器指令碼
9. 🚀 使用負載測試檢查 Cloud Run 自動調度資源
10. 🚀 逐步發布新修訂版本
11. 🚀 ADK 追蹤
12. 🎯 挑戰
13. 🧹 清理

1. 簡介

本教學課程將引導您在 Google Cloud Run 部署、管理及監控以 [Agent Development Kit \(ADK\)](#) 建構的強大代理。ADK 可協助您建立能夠執行複雜多代理工作流程的代理。只要運用全代管的無伺服器平台 Cloud Run，就能將代理程式部署為可擴充的容器化應用程式，不必擔心基礎架構問題。這項強大組合可讓您專注於代理程式的核心邏輯，同時享有 Google Cloud 穩固且可擴充的環境。

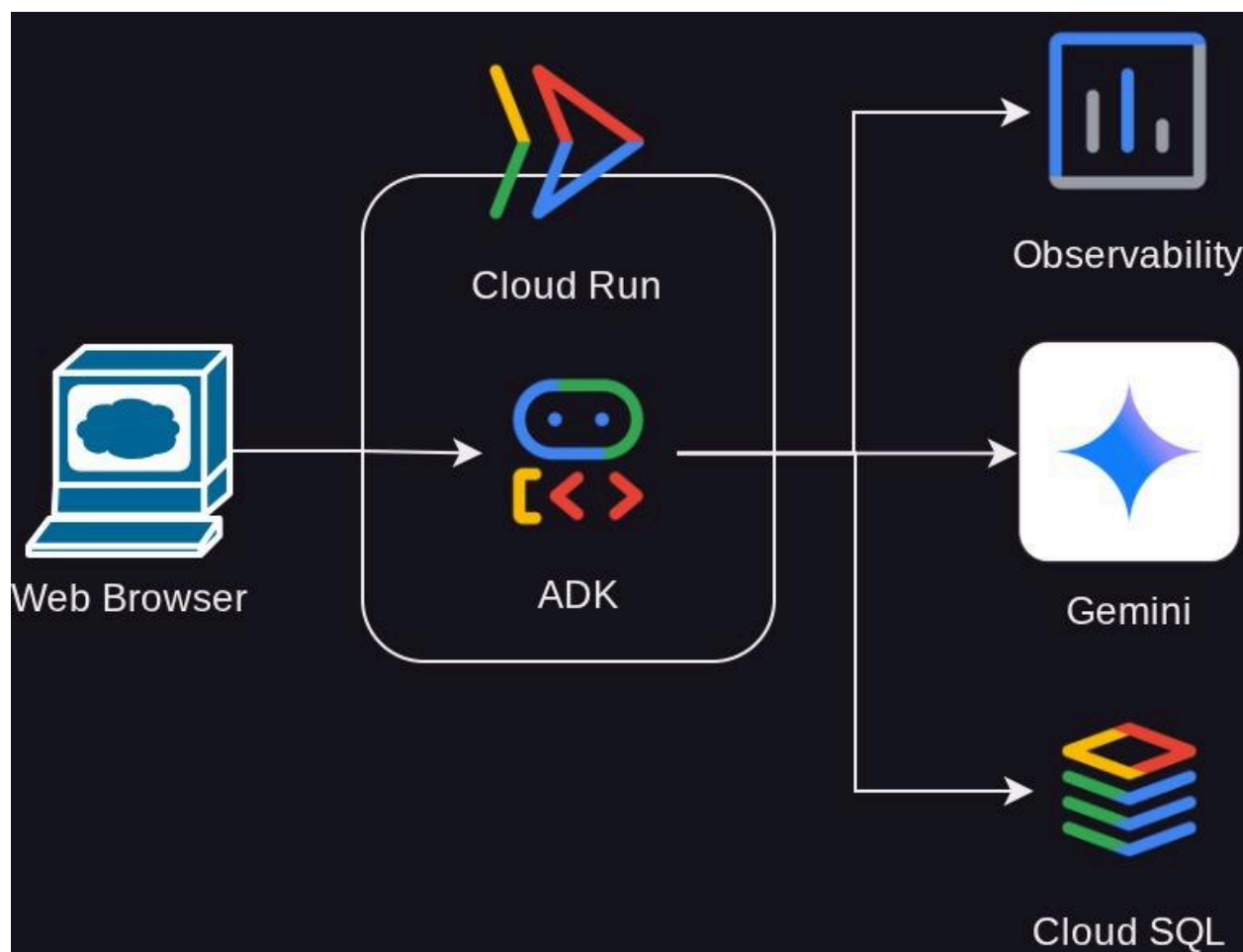
在本教學課程中，我們將探討如何將 ADK 與 Cloud Run 完美整合。您將瞭解如何部署代理程式，然後深入探討在類似於正式環境的設定中管理應用程式的實務層面。我們會說明如何管理流量，安全地推出新版代理程式，讓您在全面發布前，先對部分使用者測試新功能。

此外，您還會實際操作，監控代理程式的成效。我們會進行負載測試，模擬實際情境，觀察 Cloud Run 的自動調整資源配置功能運作情形。為深入瞭解代理程式的行為和成效，我們將透過 Cloud Trace 啟用追蹤功能。這項功能會提供要求在代理程式中傳遞的端對端詳細檢視畫面，方便您找出並解決任何效能瓶頸。完成本教學課程後，您將全面瞭解如何在 Cloud Run 上有效部署、管理及監控 ADK 輔助的代理程式。

在本程式碼研究室中，您將逐步完成下列步驟：

1. 在 CloudSQL 上建立 PostgreSQL 資料庫，用於 ADK Agent 資料庫工作階段服務
2. 設定基本 ADK 代理
3. 設定 ADK 執行器使用的資料庫工作階段服務
4. 將代理程式初步部署至 Cloud Run
5. 進行負載測試，並檢查 Cloud Run 自動調度資源功能
6. 部署新的代理程式修訂版本，並逐步增加新修訂版本的流量
7. 設定雲端追蹤功能，並檢查代理程式執行追蹤記錄

架構總覽



必要條件

- 熟悉 Python
- 瞭解使用 HTTP 服務的基本全端架構

課程內容

- ADK 結構和本機公用程式
- 使用資料庫工作階段服務設定 ADK 代理
- 在 CloudSQL 中設定 PostgreSQL，供資料庫工作階段服務使用
- 使用 Dockerfile 將應用程式部署至 Cloud Run，並設定初始環境變數
- 透過負載測試設定及測試 Cloud Run 自動調度資源功能
- 使用 Cloud Run 逐步發布的策略
- 將 ADK 代理追蹤記錄設定為 Cloud Trace

軟硬體需求

- Chrome 網路瀏覽器
- Gmail 帳戶
- 已啟用計費功能的 Cloud 專案

本程式碼研究室適合各種程度的開發人員 (包括初學者)，並使用 Python 撰寫範例應用程式。不過，您不需要具備 Python 知識，也能瞭解本文介紹的概念。

步驟 2 · 約 0 分鐘

2. 🚀 準備研討會設定

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

目標：確認專案和帳單帳戶已準備就緒、熟悉 Cloud Shell 終端機和編輯器，並設定工作目錄和基本代理程式

重要注意事項：

本教學課程需要具備有效帳單帳戶的 Google Cloud 專案。如果沒有，請使用本程式碼研究室頂端的橫幅，取得可用於本教學課程的試用帳單帳戶。

現在，我們將在本教學課程中使用 Cloud Shell IDE，請點選下列按鈕前往該處

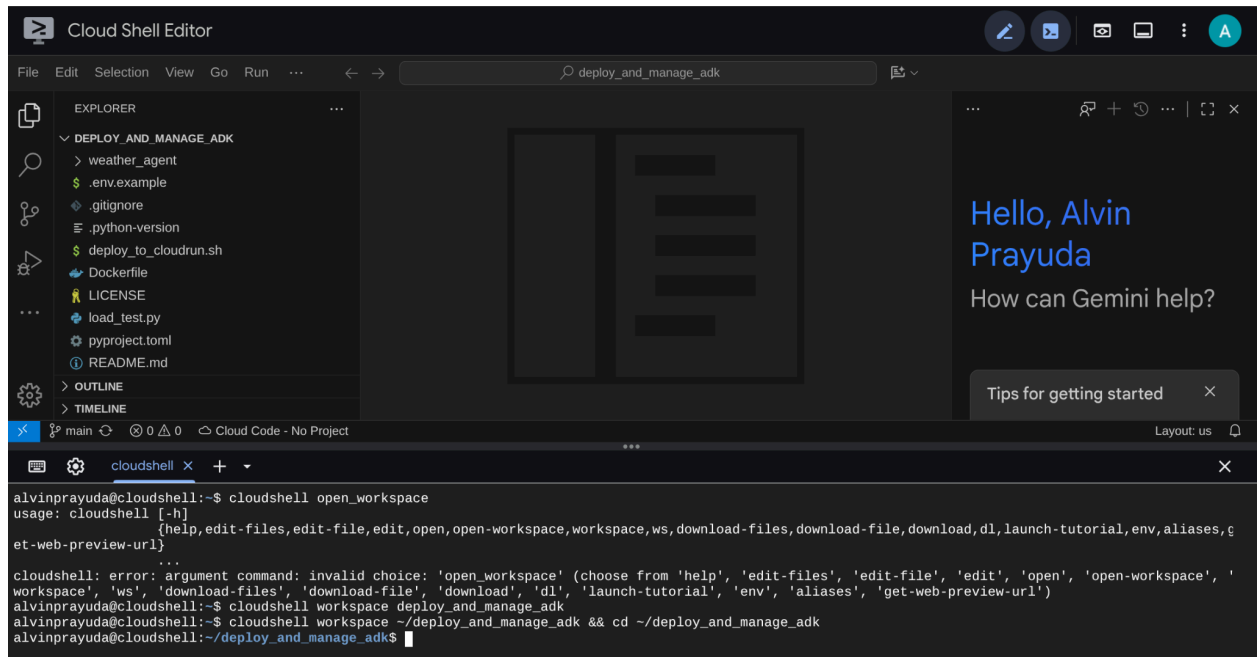
進入 Cloud Shell 後，請從 GitHub 複製本程式碼研究室的範本工作目錄，方法是執行下列指令。系統會在 **deploy_and_manage_adk** 目錄中建立工作目錄

```
git clone https://github.com/alphinside/deploy-and-manage-adk-service.git
deploy_and_manage_adk
```

接著，在終端機中執行下列指令，開啟複製的存放區做為工作目錄

```
cloudshell workspace ~/deploy_and_manage_adk && cd ~/deploy_and_manage_adk
```

完成後，介面應如下所示



這就是我們的主要介面，頂端是 IDE，底部是終端機。現在我們需要準備終端機，建立並啟用 Google Cloud 專案，該專案會連結至先前申請的試用帳單帳戶。我們已準備好指令碼，確保終端機工作階段隨時就緒。執行下列指令（請確認您已在 `deploy_and_manage_adk` 工作區中）：

```
bash setup_trial_project.sh && source .env
```

執行這項操作時，系統會提示建議的專案 ID 名稱，您可以按下 `Enter` 繼續

```
Step 2: Create a new GCP Project
Suggested project ID: deploy-manage-adk-7f5f84d38e2c
Enter project ID (press Enter for suggested):
```

稍待片刻後，如果主控台顯示以下輸出內容，即可前往下一個步驟

```
=====
Setup Complete! 🎉
=====
Project ID:      deploy-manage-adk-7f5f84d38e2c
Billing Account: 017EE1-E9B1F4-009DA0
Environment:    .env
=====

You can now proceed with the workshop!
To verify, run: gcloud config get-value project
alvinprayuda@cloudshell:~/deploy_and_manage_adk (deploy-manage-adk-7f5f84d38e2c)$
```

這表示終端機已通過驗證，並設為正確的專案 ID（目前目錄路徑旁的黃色文字）。這項指令可協助您建立新專案、找出專案並連結至試用帳單帳戶、準備環境變數設定的 `.env` 檔案，以及在終端機中啟用正確的專案 ID。

現在，我們準備好進行下一個步驟

3. 🚀 啟用 API

目標：確保已啟用必要的 API

在本教學課程中，我們會與 Cloud SQL 資料庫、Gemini 模型和 Cloud Run 互動，這些產品需要啟用下列 API，請執行這些指令來啟用：

這可能需要一些時間。

```
gcloud services enable aiplatform.googleapis.com \  
                        run.googleapis.com \  
                        cloudbuild.googleapis.com \  
                        cloudresourcemanager.googleapis.com \  
                        sqladmin.googleapis.com \  
                        compute.googleapis.com
```

成功執行指令後，您應該會看到類似下方的訊息：

```
Operation "operations/..." finished successfully.
```


步驟 4 · 約 0 分鐘

4. 🚀 Python 環境設定和環境變數

在本程式碼研究室中，我們將使用 Python 3.12，並使用 [uv Python 專案管理工具](#)，簡化建立及管理 Python 版本和虛擬環境的需求。Cloud Shell 已預先安裝 **uv** 套件。

注意：使用 **uv** 做為 Python 專案管理員時，建立及啟用**虛擬環境**的程序會簡化。不過，每個 **python** 指令碼指令都會替換為 **uv run** 指令。例如：**uv run main.py**，而不是 **python main.py**

執行此指令，將必要的依附元件安裝至 **.venv** 目錄的虛擬環境

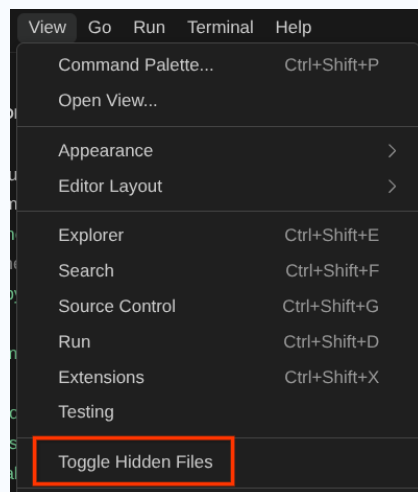
```
uv sync --frozen
```

注意：Python 依附元件會在 **pyproject.toml** 中宣告，並在 **uv.lock** 檔案中鎖定版本

接著，我們會檢查這個專案所需的環境變數檔案。先前這個檔案是由 `setup_trial_project.sh` 指令碼設定。執行下列指令，在編輯器中開啟 **.env** 檔案

```
cloudshell open .env
```

注意：如果編輯器未顯示 **.env** 檔案，表示該檔案屬於隱藏的檔案。依序點選「View」->「Toggle Hidden Files」即可顯示



您會看到 **.env** 檔案中已套用下列設定。

```
# .env

# Google Cloud and Vertex AI configuration
GOOGLE_CLOUD_PROJECT=your-project-id
GOOGLE_CLOUD_LOCATION=global
GOOGLE_GENAI_USE_VERTEXAI=True

# Database connection for session service
# DB_CONNECTION_NAME=your-db-connection-name
```

在本程式碼研究室中，我們將使用 `GOOGLE_CLOUD_LOCATION` 和 `GOOGLE_GENAI_USE_VERTEXAI` 的預先設定值。

現在我們可以前往下一個步驟，建立代理程式要使用的資料庫，以保存狀態和工作階段。

5. 🚀 準備 Cloud SQL 資料庫

目標：準備資料庫，供 ADK 代理使用

稍後 ADK 代理程式會用到資料庫，我們將在 Cloud SQL 中建立 PostgreSQL 資料庫。請先執行下列指令，建立資料庫執行個體。我們將使用預設的 **postgres** 資料庫名稱，因此這裡會略過資料庫建立作業。我們也需要設定預設資料庫使用者名稱 (同樣是 **postgres**)，為了方便教學，我們將 **ADK-deployment123** 設為密碼

```
gcloud sql instances create adk-deployment \  
  --database-version=POSTGRES_17 \  
  --edition=ENTERPRISE \  
  --tier=db-g1-small \  
  --region=us-west1 \  
  --availability-type=ZONAL \  
  --project=${GOOGLE_CLOUD_PROJECT} && \  
gcloud sql users set-password postgres \  
  --instance=adk-deployment \  
  --password=ADK-deployment123
```

警告！ 建立新的 Cloud SQL 執行個體大約需要 **5 到 7 分鐘**。請耐心等待，**不要關閉或終止終端機**

重要事項！ 使用者名稱和密碼屬於私密資料，請務必在實際工作環境中對他人保密！

在上述指令中，第一個常見的 `gcloud sql instances create adk-deployment` 是用來建立資料庫執行個體的指令。在本教學課程中，我們將使用沙箱最低規格。第二個指令 `gcloud sql users set-password postgres` 用於變更預設 **postgres** 使用者名稱密碼

請注意，我們使用 **adk-deployment** 做為資料庫執行個體名稱。完成後，終端機應會顯示如下所示的輸出內容，表示執行個體已準備就緒，且預設使用者密碼已更新

```
Created [https://sqladmin.googleapis.com/sql/v1beta4/projects/your-project-id/instances/adk-deployment].  
NAME: adk-deployment  
DATABASE_VERSION: POSTGRES_17  
LOCATION: us-west1-a  
TIER: db-g1-small  
PRIMARY_ADDRESS: xx.xx.xx.xx  
PRIVATE_ADDRESS: -  
STATUS: RUNNABLE  
Updating Cloud SQL user...done.
```

部署這個資料庫需要一些時間，在等待 Cloud SQL 資料庫部署完成期間，請繼續前往下一節。

警告！再次提醒，請勿關閉或終止終端機

步驟 6 · 約 0 分鐘

6. 🚀 使用 ADK 和 Gemini 2.5 建構天氣代理

ADK 目錄結構簡介

首先，我們來瞭解 ADK 的功能，以及如何建構代理程式。如要查看 ADK 完整說明文件，請前往[這個網址](#)。ADK 在執行 CLI 指令時提供許多公用程式。部分範例如下：

- 設定代理程式目錄結構
- 透過 CLI 輸入/輸出快速試用互動功能
- 快速設定本機開發 UI 網頁介面

現在，我們來檢查 **weather_agent** 目錄中的代理程式結構

```
weather_agent/  
├── __init__.py  
├── agent.py  
└── tool.py
```

檢查 **init.py** 和 **agent.py** 時，您會看到這段程式碼

```
# __init__.py  
  
from weather_agent.agent import root_agent  
  
__all__ = ["root_agent"]
```

```
# agent.py

import os
from pathlib import Path

import google.auth
from dotenv import load_dotenv
from google.adk.agents import Agent
from weather_agent.tool import get_weather

# Load environment variables from .env file in root directory
root_dir = Path(__file__).parent.parent
dotenv_path = root_dir / ".env"
load_dotenv(dotenv_path=dotenv_path)

# Use default project from credentials if not in .env
_, project_id = google.auth.default()
os.environ.setdefault("GOOGLE_CLOUD_PROJECT", project_id)
os.environ.setdefault("GOOGLE_CLOUD_LOCATION", "global")
os.environ.setdefault("GOOGLE_GENAI_USE_VERTEXAI", "True")

root_agent = Agent(
    name="weather_agent",
    model="gemini-2.5-flash",
    instruction="""
You are a helpful AI assistant designed to provide accurate and useful information.
""",
    tools=[get_weather],
)
```

注意：ADK CLI 必須能辨識目錄結構、**init.py** 內的代理匯入項目，以及 **agent.py** 中的 **root_agent** 宣告，因此這些都是必要結構。

ADK 程式碼說明

這個指令碼包含代理程式啟動程序，我們會初始化下列項目：

- 設定用於 `gemini-2.5-flash` 的模型
- 提供工具 `get_weather`，支援代理程式功能 (例如天氣代理程式)

在本機執行網頁版 UI

現在，我們可以與代理程式互動，並在本機檢查其行為。ADK 可讓我們透過開發網頁 UI 互動，並檢查互動期間發生的情況。執行下列指令，啟動本機開發 UI 伺服器

```
uv run adk web --port 8080
```

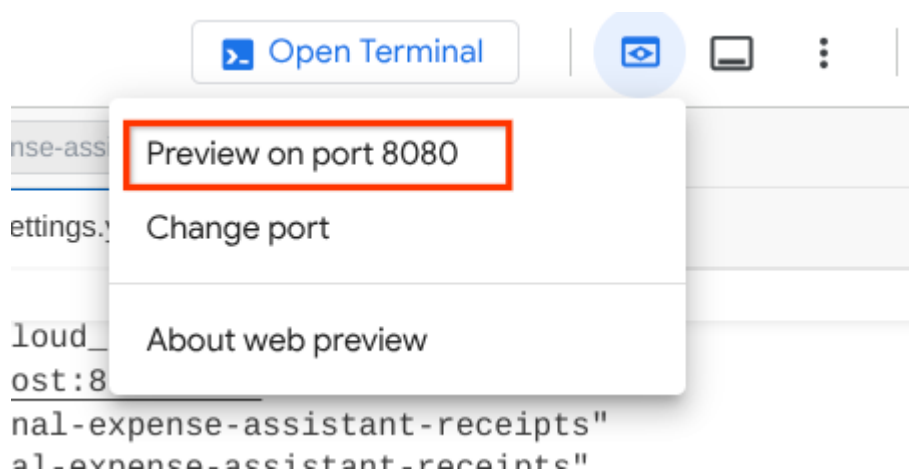
這會產生類似下列範例的輸出內容，表示我們已可存取網頁介面

```
INFO:      Started server process [xxxx]
INFO:      Waiting for application startup.

+-----+
| ADK Web Server started                                |
|                                                       |
| For local testing, access at http://localhost:8080.  |
+-----+

INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
```

如要檢查，請點選 Cloud Shell 編輯器頂端的「Web Preview」(網頁預覽) 按鈕，然後選取「Preview on port 8080」(透過以下通訊埠預覽：8080)。



您會看到下列網頁，可以在左上方的下拉式按鈕中選取可用的代理程式 (在本例中應為 **weather_agent**)，並與機器人互動。在左側視窗中，您會看到代理程式執行階段的記錄詳細資料

Agent Development Kit

weather_a... ▾

<

Trace

Events

State

>

Invocations

hello ▾

show me weather in new York p ▾

SESSION ID 9446b7dc-0a42-41c9-b6d5-9ba8add667ba

Token Streaming

+ New Session

hello

Hello! How can I help you today?

show me weather in new York pls

get_weather

✓ get_weather

The weather in New York is sunny with a temperature of 25°C.

Type a Message...

現在試著與其互動。在左側列中，我們可以檢查每個輸入內容的追蹤記錄，瞭解代理程式在形成最終答案前，執行每個動作所需的時間。

The screenshot displays the Agent Development Kit (ADK) interface. On the left, a sidebar shows the 'weather_a...' dropdown, tabs for 'Trace', 'Events', and 'State', and a list of 'Invocations'. The selected invocation is 'hello', which is expanded to show a detailed trace for 'show me weather in new York p...'. The trace includes the following steps and durations:

- Invocation ID: e-94c61f6f-dd04-430d-aa1c-6de1c2e242e6
- invocation: 2487.03ms
- agent_run...: 2486.54ms
- call_llm: 1238.07ms
- exe...: 1140.50ms
- call_llm: 1246.43ms

The main chat area on the right shows a session with ID '9446b7dc-0a42-41c9-b6d5-9ba8add667ba'. It includes a 'Token Streaming' toggle and buttons for 'New Session', 'Trash', and 'Download'. The chat history shows a user saying 'hello', an assistant responding 'Hello! How can I help you today?', the user asking 'show me weather in new York pls', the assistant calling 'get_weather', and finally providing the weather information: 'The weather in New York is sunny with a temperature of 25°C.' At the bottom, there is a 'Type a Message...' input field with icons for attachments, voice, and video.

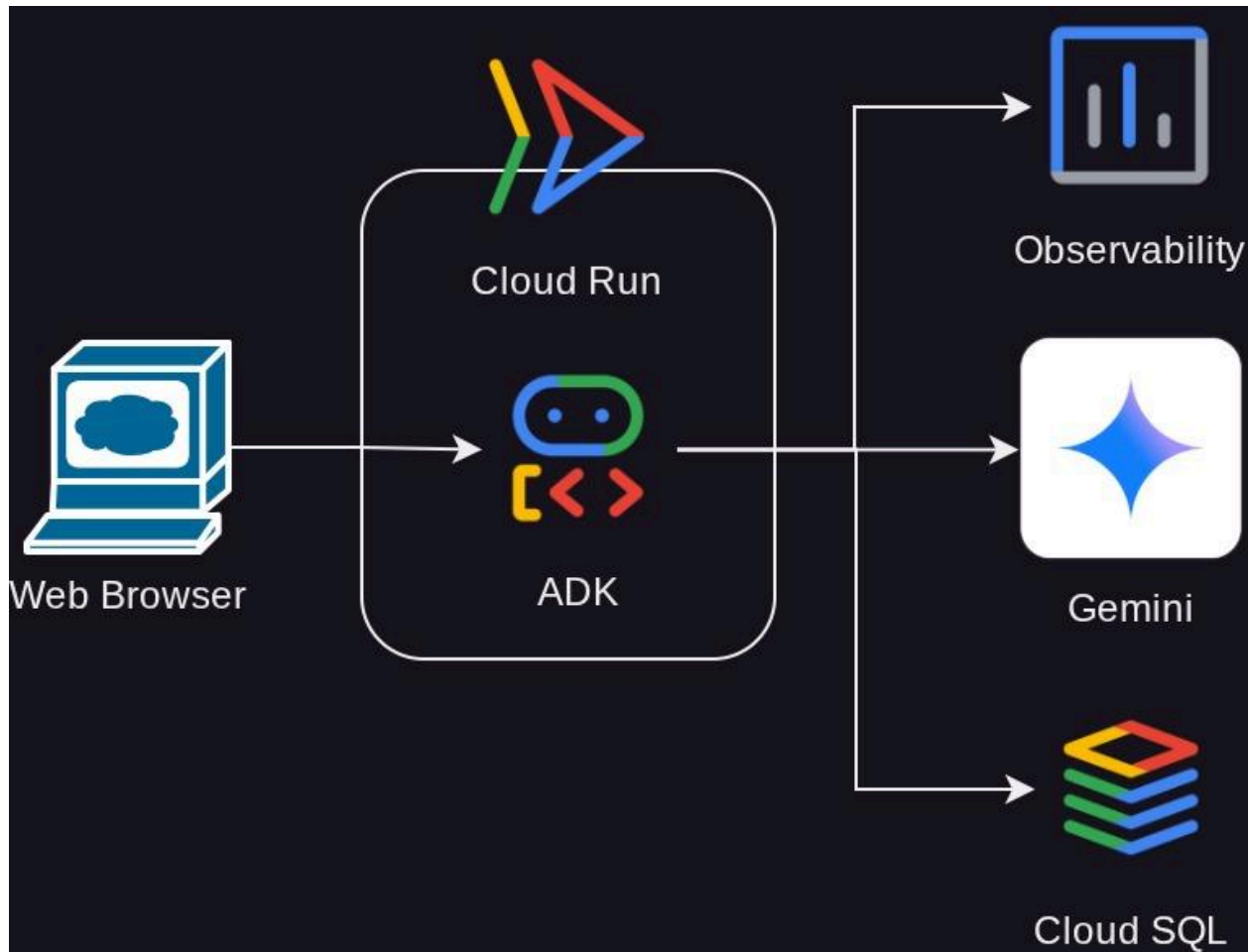
這是 ADK 內建的可觀測性功能之一，目前我們在本機檢查這項功能。稍後我們將瞭解如何將這項功能整合至 Cloud Tracing，集中追蹤所有要求

提醒事項！現在先終止 `adk web` 程序，按下 **Ctrl + C** 即可終止程序

步驟 7 · 約 0 分鐘

7. 🚀 部署至 Cloud Run

現在，請將這個代理程式服務部署至 Cloud Run。為了進行這項示範，這項服務會公開，供其他人存取。不過請注意，這並非最佳做法，因為不安全



這個部署情境可讓您自訂代理程式後端服務，我們將使用 Dockerfile 將代理程式部署至 Cloud Run。此時，我們已備妥將應用程式部署至 Cloud Run 所需的所有檔案 (**Dockerfile** 和 **server.py**)。有了這 2 個項目，您就能彈性自訂代理程式部署作業 (例如新增自訂後端路徑，以及/或是為了監控目的新增額外的 Sidecar 服務)。我們稍後會詳細討論這項主題。

現在，請先部署服務，然後前往 Cloud Shell 終端機，確認目前的專案已設為現用專案，接著再次執行設定指令碼。您也可以視需要使用 `gcloud config set project [PROJECT_ID]` 指令設定現用專案

```
bash setup_trial_project.sh && source .env
```

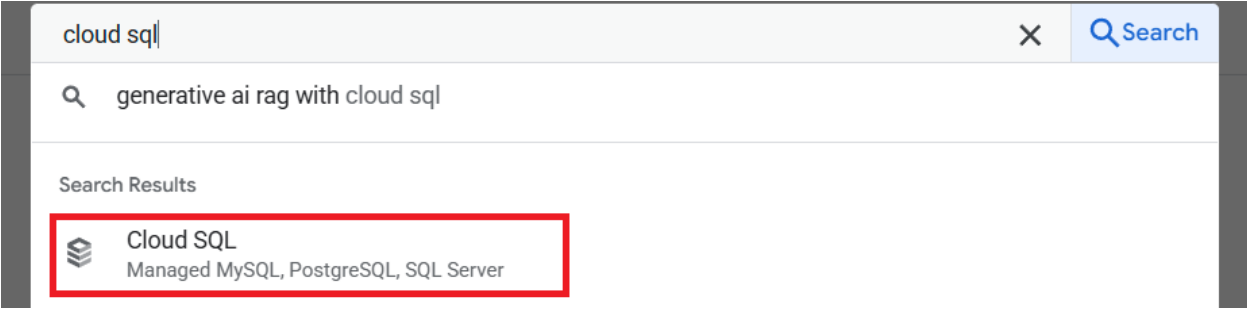
現在我們需要再次開啟 `.env` 檔案，您會看到需要取消註解 `DB_CONNECTION_NAME` 變數，並填入正確值

```
# Google Cloud and Vertex AI configuration
GOOGLE_CLOUD_PROJECT=your-project-id
GOOGLE_CLOUD_LOCATION=global
GOOGLE_GENAI_USE_VERTEXAI=True

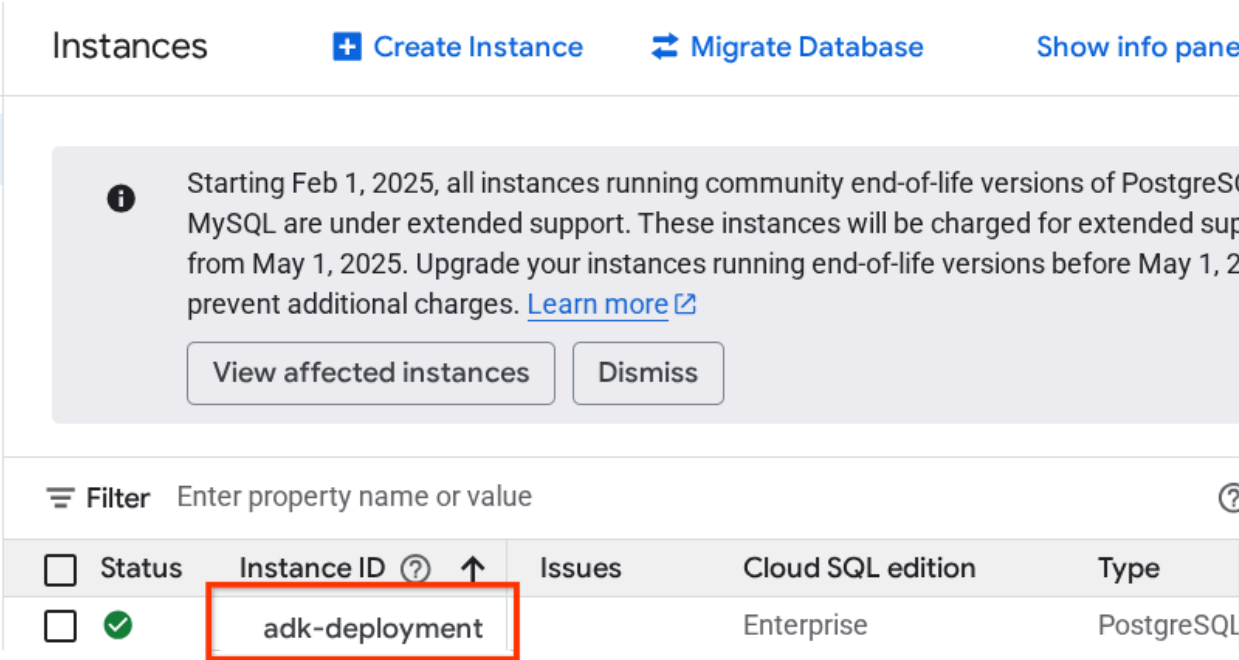
# Database connection for session service
DB_CONNECTION_NAME=your-db-connection-name
```

如要取得 **DB_CONNECTION_NAME** 值，請前往 Cloud SQL 資訊主頁

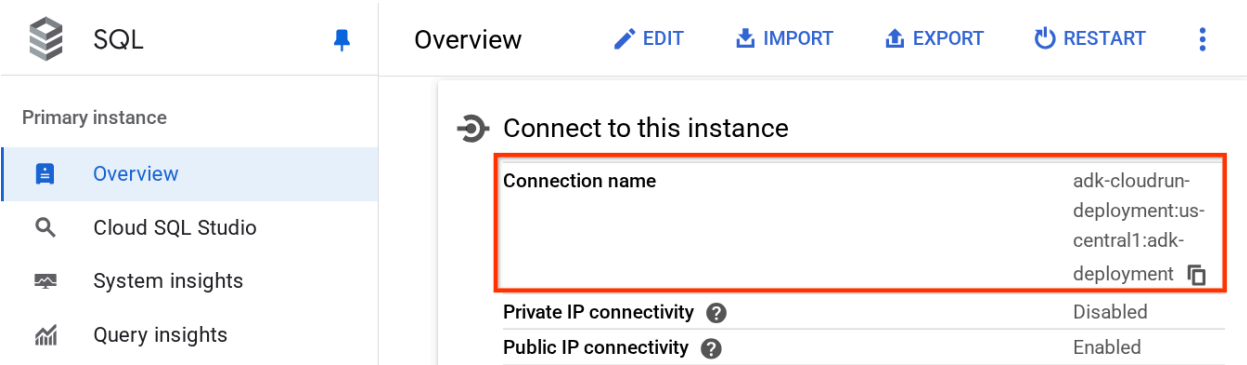
然後點選您建立的執行個體。前往 Cloud 控制台頂端區段的搜尋列，然後輸入「Cloud SQL」。然後按一下「Cloud SQL」產品。



接著，您會看到先前建立的執行個體，請點選該執行個體



在執行個體頁面中，向下捲動至「連線至這個執行個體」部分，即可複製「連線名稱」，以取代 **DB_CONNECTION_NAME** 值。



然後執行下列指令，開啟 **.env** 檔案

```
cloudshell edit .env
```

並修改 **.env** 檔案中的 **DB_CONNECTION_NAME** 變數。您的 env 檔案應如下列範例所示

```
# Google Cloud and Vertex AI configuration
GOOGLE_CLOUD_PROJECT=your-project-id
GOOGLE_CLOUD_LOCATION=global
GOOGLE_GENAI_USE_VERTEXAI=True

# Database connection for session service
DB_CONNECTION_NAME=your-project-id:your-location:your-instance-name
```

然後執行部署指令碼

```
bash deploy_to_cloudrun.sh
```

如果系統提示您確認要為 Docker 存放區建立 Artifact Registry，請回答 **Y**。

等待部署程序完成的同時，讓我們來看看 **deploy_to_cloudrun.sh**。

```
#!/bin/bash

# Load environment variables from .env file
if [ -f .env ]; then
    export $(cat .env | grep -v '^#' | xargs)
else
    echo "Error: .env file not found"
    exit 1
fi

# Validate required variables
required_vars=("GOOGLE_CLOUD_PROJECT" "DB_CONNECTION_NAME")
for var in "${required_vars[@]}; do
    if [ -z "${!var}" ]; then
        echo "Error: $var is not set in .env file"
        exit 1
    fi
done

gcloud run deploy weather-agent \
    --source . \
    --port 8080 \
    --project ${GOOGLE_CLOUD_PROJECT} \
    --allow-unauthenticated \
    --add-cloudsql-instances ${DB_CONNECTION_NAME} \
    --update-env-vars SESSION_SERVICE_URI="postgresql+pg8000://postgres:ADK-
deployment123@postgres/?
unix_sock=/cloudsql/${DB_CONNECTION_NAME}/.s.PGSQL.5432",GOOGLE_CLOUD_PROJECT=${GOOGLE_CLOUD_
PROJECT} \
    --region us-west1 \
    --min 1 \
    --memory 1G \
    --concurrency 10
```

這個指令碼會載入 **.env** 變數，然後執行部署指令。

仔細觀察後，您會發現我們只需要一個 **gcloud run deploy** 指令，就能完成部署服務所需的所有必要事項：建構映像檔、推送至註冊資料庫、部署服務、設定 IAM 政策、建立修訂版本，甚至是轉送流量。在本範例中，我們已提供 Dockerfile，因此這項指令會使用該檔案建構應用程式

注意：在部署指令碼中，我們會設定 **SESSION_SERVICE_URI** 環境變數。如果設定了這項變數，ADK 代理程式就會使用服務的 URI 儲存工作階段資訊和狀態 (定義於 **server.py** 中)。在本教學課程中，我們將使用 **pg8000** 驅動程式，透過 PostgreSQL Unix 通訊端連線。

請注意，在範例中，我們使用預設使用者 **postgres** 和預設資料庫 **postgres** (密碼為 **ADK-deployment123**)。

重要事項！由於這是示範應用程式，因此我們允許未經驗證的存取要求 (旗標 **-allow_unauthenticated**)。建議您為企業和正式版應用程式使用適當的驗證方式。

部署完成後，您會取得類似下方的連結：

[https://weather-agent-***.us-west1.run.app](https://weather-agent-*****.us-west1.run.app)**

取得這個網址後，您就可以在無痕視窗或行動裝置上使用應用程式，並存取代理程式開發人員使用者介面。等待部署期間，請前往下一節檢查我們剛部署的詳細服務

步驟 8 · 約 0 分鐘

8. 💡 Dockerfile 和後端伺服器指令碼

為了讓代理程式可做為服務存取，我們會將代理程式包裝在 FastAPI 應用程式中，並在 Dockerfile 指令中執行。以下是 **Dockerfile** 的內容

```
FROM python:3.12-slim

RUN pip install --no-cache-dir uv==0.7.13

WORKDIR /app

COPY . .

RUN uv sync --frozen

EXPOSE 8080

CMD ["uv", "run", "uvicorn", "server:app", "--host", "0.0.0.0", "--port", "8080"]
```

我們可以在這裡設定必要服務來支援代理程式，例如準備用於生產環境的[工作階段](#)、[記憶體](#)或[構件](#)服務。以下是將使用的 **server.py** 程式碼

```

import os

from dotenv import load_dotenv
from fastapi import FastAPI
from google.adk.cli.fast_api import get_fast_api_app
from pydantic import BaseModel
from typing import Literal
from google.cloud import logging as google_cloud_logging

# Load environment variables from .env file
load_dotenv()

logging_client = google_cloud_logging.Client()
logger = logging_client.logger(__name__)

AGENT_DIR = os.path.dirname(os.path.abspath(__file__))

# Get session service URI from environment variables
session_uri = os.getenv("SESSION_SERVICE_URI", None)

# Prepare arguments for get_fast_api_app
app_args = {"agents_dir": AGENT_DIR, "web": True, "trace_to_cloud": True}

# Only include session_service_uri if it's provided
if session_uri:
    app_args["session_service_uri"] = session_uri
else:
    logger.log_text(
        "SESSION_SERVICE_URI not provided. Using in-memory session service instead. "
        "All sessions will be lost when the server restarts.",
        severity="WARNING",
    )

# Create FastAPI app with appropriate arguments
app: FastAPI = get_fast_api_app(**app_args)

app.title = "weather-agent"
app.description = "API for interacting with the Agent weather-agent"

class Feedback(BaseModel):
    """Represents feedback for a conversation."""

    score: int | float
    text: str | None = ""
    invocation_id: str
    log_type: Literal["feedback"] = "feedback"
    service_name: Literal["weather-agent"] = "weather-agent"
    user_id: str = ""

```

```
# Example if you want to add your custom endpoint
@app.post("/feedback")
def collect_feedback(feedback: Feedback) -> dict[str, str]:
    """Collect and log feedback.

    Args:
        feedback: The feedback data to log

    Returns:
        Success message
    """
    logger.log_struct(feedback.model_dump(), severity="INFO")
    return {"status": "success"}

# Main execution
if __name__ == "__main__":
    import uvicorn

    uvicorn.run(app, host="0.0.0.0", port=8080)
```

伺服器程式碼說明

這些是 **server.py** 指令碼中定義的項目：

1. 使用 `get_fast_api_app` 方法將代理程式轉換為 FastAPI 應用程式。這樣一來，我們就會沿用用於網頁開發 UI 的相同路徑定義。
2. 將關鍵字引數新增至 `get_fast_api_app` 方法，設定必要的工作階段、記憶體或構件服務。在本教學課程中，如果我們設定 `SESSION_SERVICE_URI` 環境變數，工作階段服務就會使用該變數，否則會使用記憶體內工作階段。
3. 我們可以新增自訂路徑，支援其他後端商業邏輯。在指令碼中，我們新增了意見回饋功能路徑範例。
4. 在 `get_fast_api_app` `arg` 參數中啟用雲端追蹤，將追蹤記錄傳送至 Google Cloud Trace。
5. 使用 `uvicorn` 執行 FastAPI 服務。

如果部署作業已完成，請存取 Cloud Run 網址，嘗試透過網頁開發人員使用者介面與代理互動。

9. 🚀 使用負載測試檢查 Cloud Run 自動調度資源

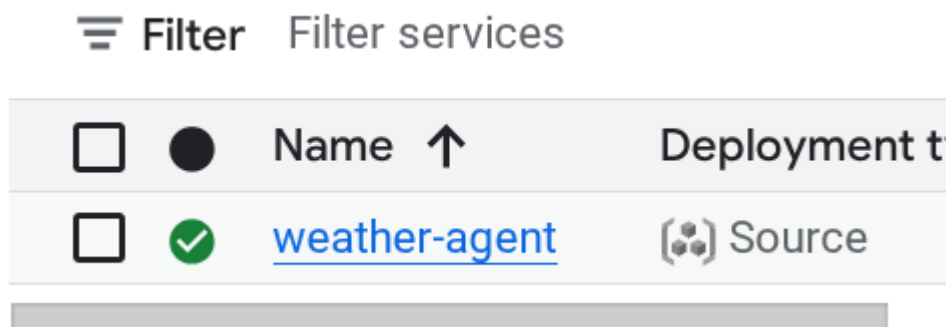
現在，我們將檢查 Cloud Run 的自動調整資源配置功能。在這個情境中，我們將部署新修訂版本，同時為每個執行個體啟用並行數上限。在前一節中，我們將並行數上限設為 10 (`--concurrency 10` 旗標)。因此，當我們進行負載測試時，如果超過這個數字，Cloud Run 就會嘗試擴充執行個體。

現在來檢查 `load_test.py` 檔案。這個指令碼會使用 `locust` 架構執行負載測試。這個指令碼會執行下列動作：

1. 隨機產生的 `user_id` 和 `session_id`
2. 為 `user_id` 建立 `session_id`
3. 使用建立的 `user_id` 和 `session_id` 存取「/run_sse」端點

如果您錯過部署的服務網址，我們需要知道該網址。我們可以前往 Cloud Run 控制台

然後找出 **weather-agent** 服務並點選



服務網址會顯示在「區域」資訊旁邊。例如：



為簡化作業，請執行下列指令碼，取得最近部署的服務網址，並儲存在 `SERVICE_URL` 環境變數中

```
export SERVICE_URL=$(gcloud run services describe weather-agent \
  --platform managed \
  --region us-west1 \
  --format 'value(status.url)')
```

然後執行下列指令，對代理程式應用程式進行負載測試

```
uv run locust -f load_test.py \
  -H $SERVICE_URL \
  -u 60 \
  -r 5 \
  -t 120 \
  --headless
```

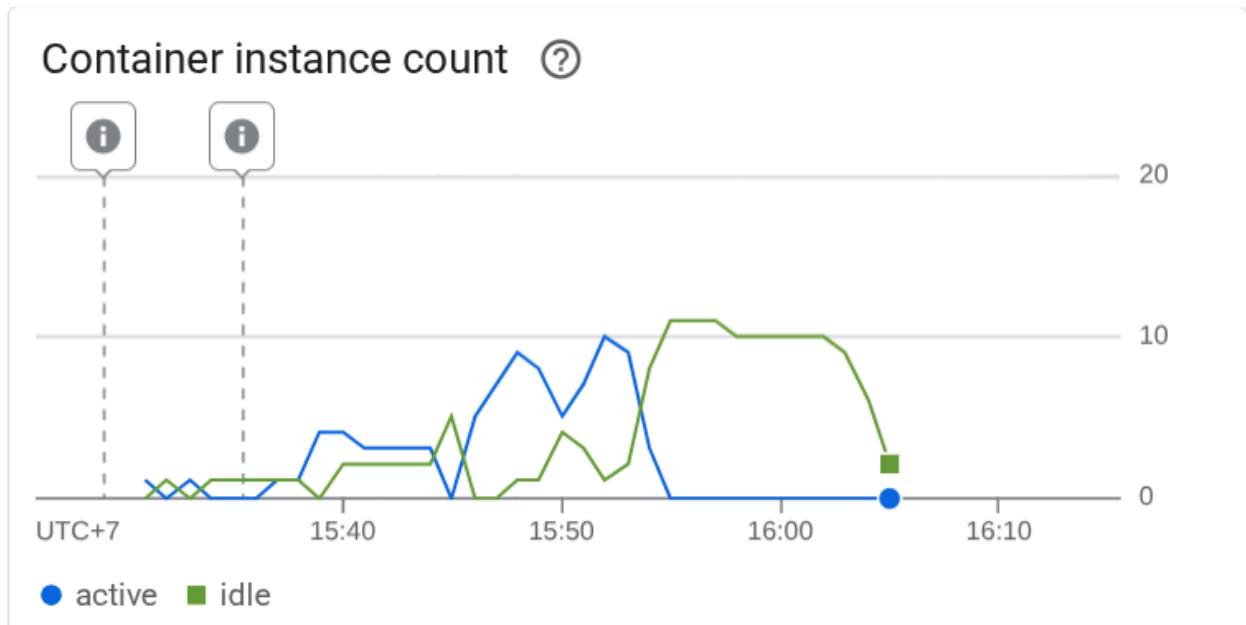
重要注意事項：請確認您已正確設定 `SERVICE_URL` 變數

注意：這會在 120 秒期間內，模擬 0 到 60 個並行使用者要求，並以每秒 5 位使用者的速率增加

執行這項操作後，您會看到類似下方的指標。(In this example all reqs success)

Type	Name		# reqs	# fails	Avg	Min
Max	Med	req/s failures/s				
POST	/run_sse end		813	0(0.00%)	5817	2217
26421	5000	6.79 0.00				
POST	/run_sse message		813	0(0.00%)	2678	1107
17195	2200	6.79 0.00				
Aggregated			1626	0(0.00%)	4247	1107
26421	3500	13.59 0.00				

接著，讓我們看看 Cloud Run 發生了什麼事，再次前往已部署的服務，然後查看資訊主頁。這會顯示 Cloud Run 如何自動調度執行個體，以處理傳入要求。由於我們將每個執行個體的並行數上限設為 10，因此 Cloud Run 執行個體會嘗試自動調整容器數量，以滿足這項條件。



注意：您不僅可以控制並行上限，也可以控制容器執行個體的下限和上限。詳情請參閱這份 [SDK 說明文件](#)

步驟 10 · 約 0 分鐘

10. 🚀 逐步發布新修訂版本

現在，我們來看看以下情境。我們想更新代理程式的提示。使用下列指令開啟 `weather_agent/agent.py`

```
cloudshell edit weather_agent/agent.py
```

並以以下程式碼覆寫：

```
# weather_agent/agent.py

import os
from pathlib import Path

import google.auth
from dotenv import load_dotenv
from google.adk.agents import Agent
from weather_agent.tool import get_weather

# Load environment variables from .env file in root directory
root_dir = Path(__file__).parent.parent
dotenv_path = root_dir / ".env"
load_dotenv(dotenv_path=dotenv_path)

# Use default project from credentials if not in .env
_, project_id = google.auth.default()
os.environ.setdefault("GOOGLE_CLOUD_PROJECT", project_id)
os.environ.setdefault("GOOGLE_CLOUD_LOCATION", "global")
os.environ.setdefault("GOOGLE_GENAI_USE_VERTEXAI", "True")

root_agent = Agent(
    name="weather_agent",
    model="gemini-2.5-flash",
    instruction="""
You are a helpful AI assistant designed to provide accurate and useful information.
You only answer inquiries about the weather. Refuse all other user query
""",
    tools=[get_weather],
)
```

接著，您想發布新修訂版本，但不希望所有要求流量都直接導向新版本。我們可以透過 Cloud Run 逐步發布。首先，我們需要部署新修訂版本，但要使用 **-no-traffic** 標記。儲存先前的代理程式指令碼，然後執行下列指令

```
gcloud run deploy weather-agent \
  --source . \
  --port 8080 \
  --project $GOOGLE_CLOUD_PROJECT \
  --allow-unauthenticated \
  --region us-west1 \
  --no-traffic
```

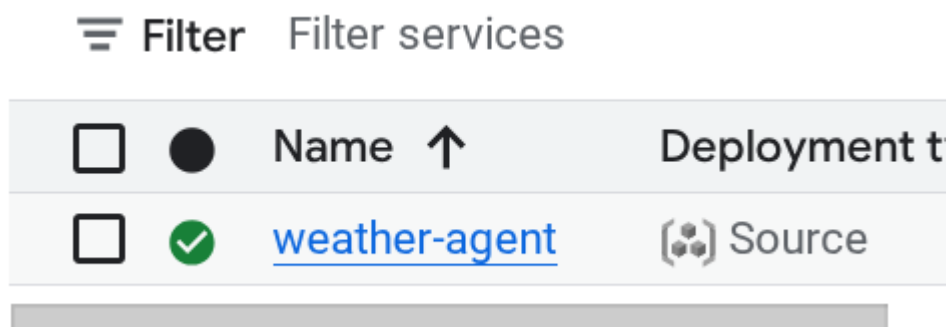
重要事項：請確認已設定 **GOOGLE_CLOUD_PROJECT** 環境變數

完成後，您會收到與先前部署程序類似的記錄，但服務的流量數不同。系統會顯示 **0%** 的流量。

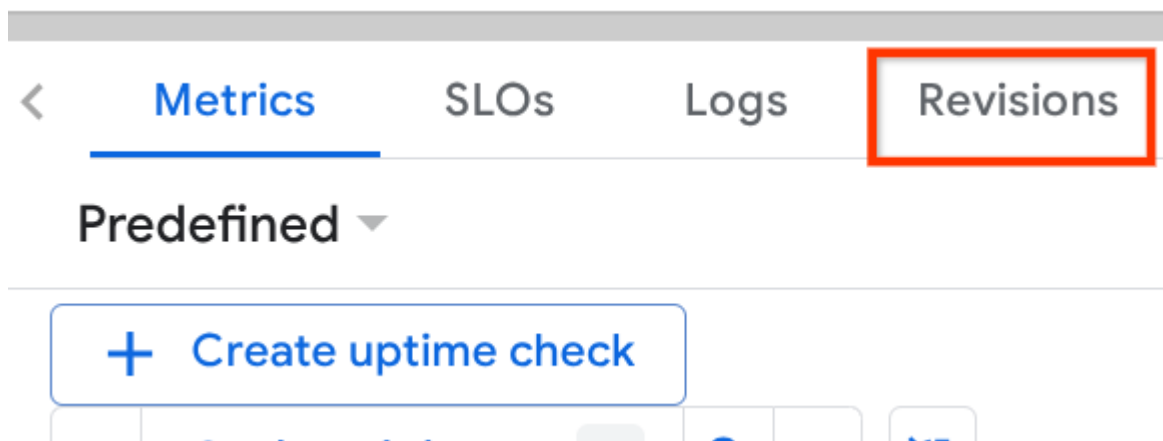
```
Service [weather-agent] revision [weather-agent-xxxx-xxx] has been deployed and is
serving 0 percent of traffic.
```

接著前往 Cloud Run 資訊主頁

然後找出 **weather-agent** 服務並點選



前往「修訂版本」分頁，即可查看已部署的修訂版本清單



您會看到新部署的修訂版本提供 0% 的服務，您可以點選 Kebab 按鈕 (:)，然後選擇「管理流量」

✓ weather-agent

Region: us-central1

URL: <https://weather-agent-29664758896>

Metrics

SLOs

Logs

Revisions

Source

Triggers

Networking

Se

Revisions

↔ Manage traffic

Filter

Filter revisions

?

☰

	Name	Traffic	Deployed ↓	Actions
⦿	weather-agent-00002-szc	0%	5 minutes ago	⋮
⦿	weather-agent-00001-qgc	100%	2 hours ago	

Delete

Manage traffic

Manage revision tags

在彈出式視窗中，您可以編輯要將多少流量導向哪個修訂版本。

Manage traffic

Revision 1 *
weather-agent-00001-qgc ▼ Serving

Revision 2 *
weather-agent-00002-szc ▼

+ Add revision

Traffic 1
90 % (Currently: 100%)

Traffic 2
10 % (Currently: 0%)

Cancel

Save

等待一段時間後，系統就會根據百分比設定，按比例分配流量。這樣一來，如果新版本發生問題，我們就能輕鬆還原至先前的修訂版本

11. 🚀 ADK 追蹤

使用 ADK 建構的代理程式已支援追蹤功能，可將開放遙測嵌入其中。我們有 Cloud Trace 可擷取這些追蹤記錄並以視覺化方式呈現。讓我們檢查 **server.py**，瞭解如何在先前部署的服務中啟用這項功能

```
# server.py

...

app_args = {"agents_dir": AGENT_DIR, "web": True, "trace_to_cloud": True}

...

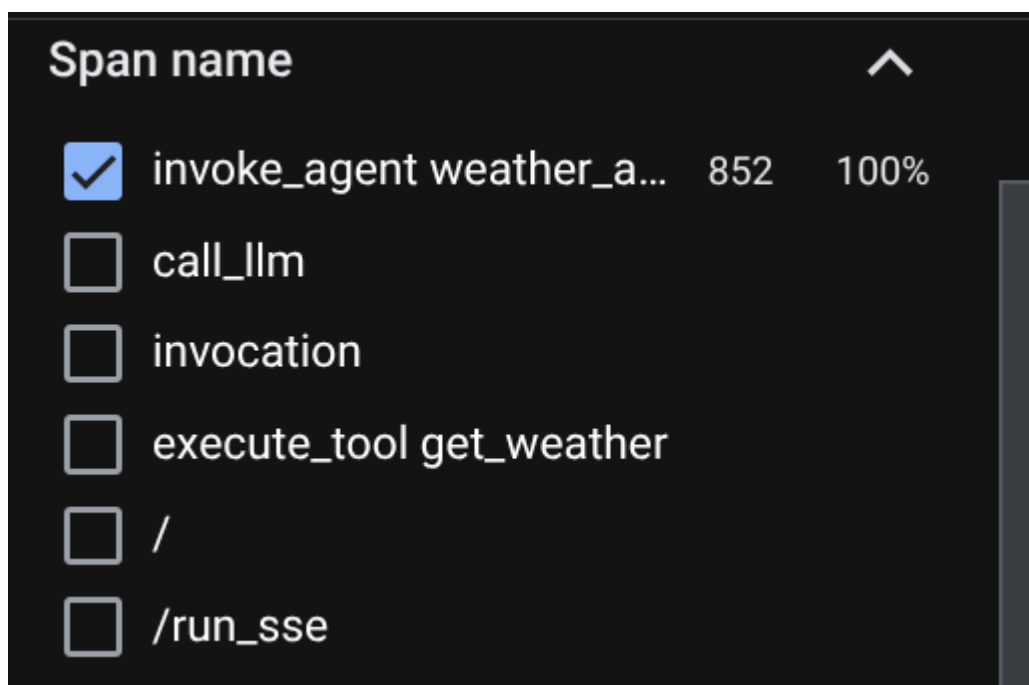
app: FastAPI = get_fast_api_app(**app_args)

...
```

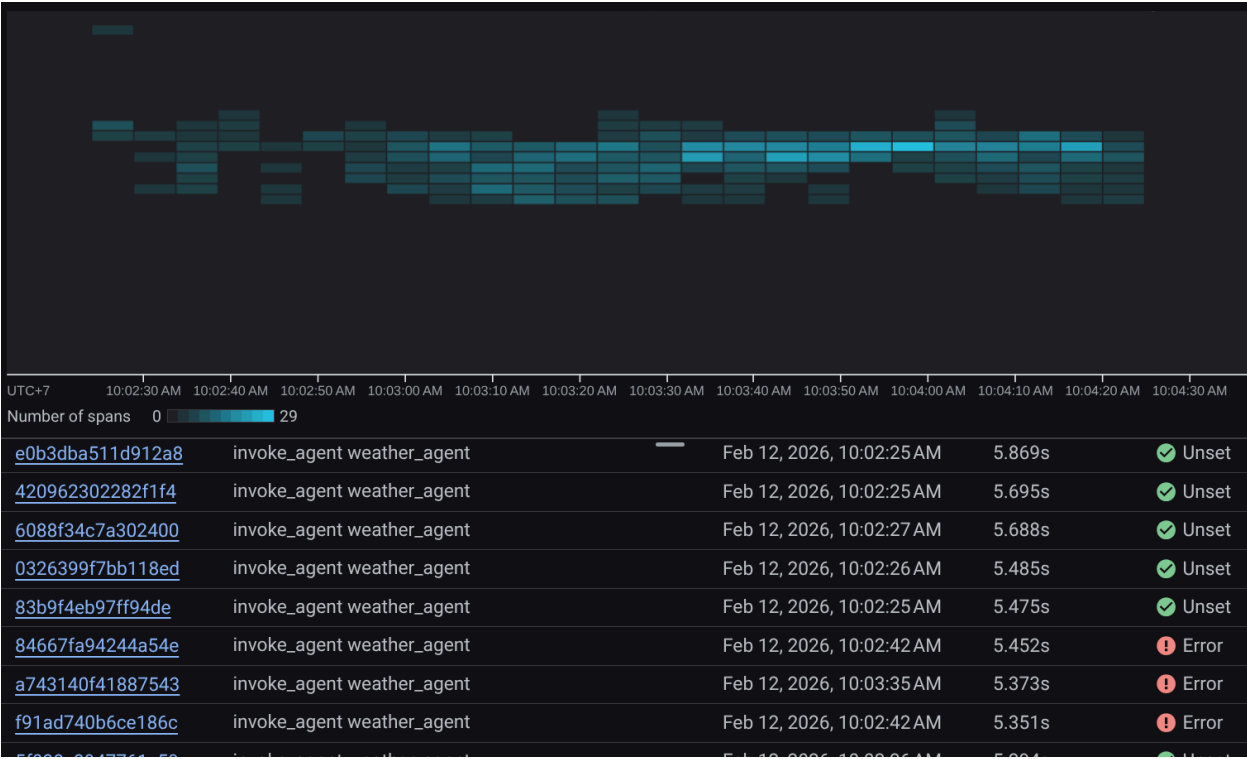
在這裡，我們將 `trace_to_cloud` 引數傳遞至 **True**。如果您使用其他選項部署，請參閱[這份文件](#)，進一步瞭解如何透過各種部署選項啟用 Cloud Trace 追蹤功能

嘗試存取服務網頁開發人員 UI，並與代理進行對話。接著前往「Trace Explorer」頁面

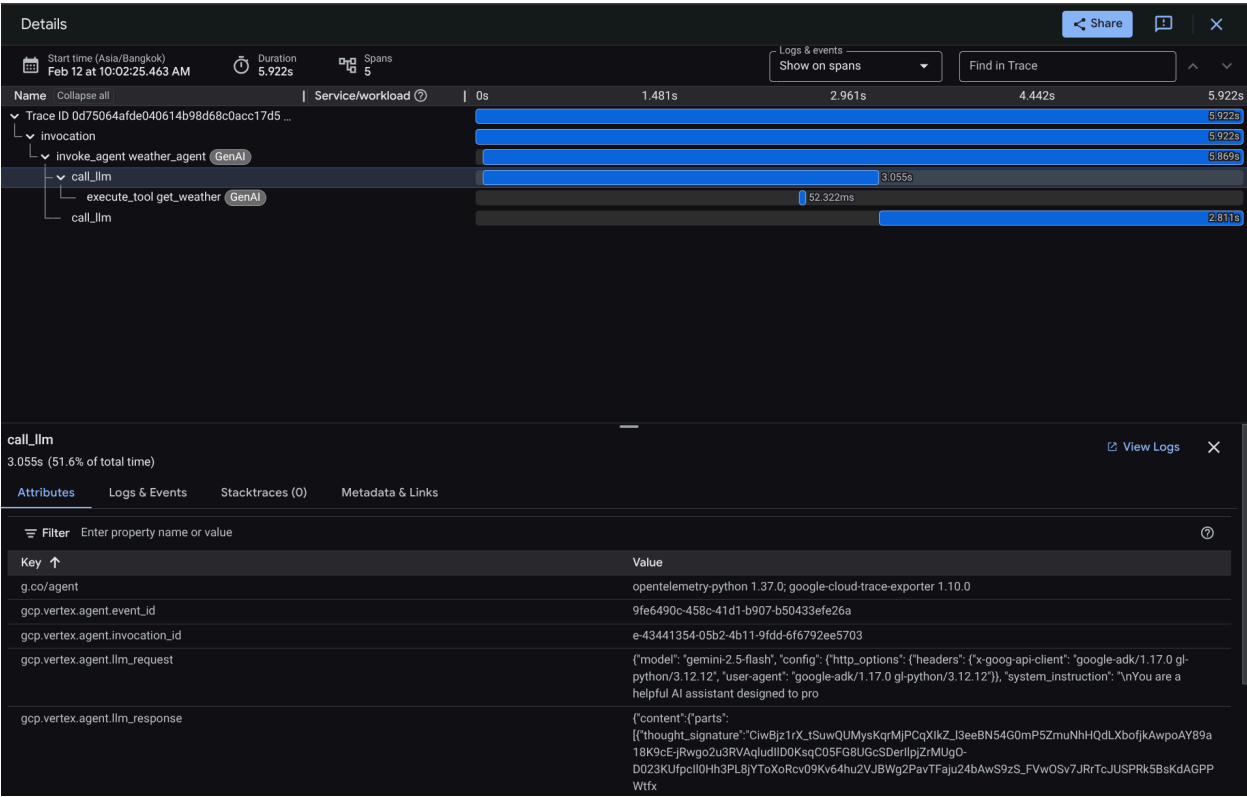
在追蹤記錄探索器頁面中，您會看到與代理程式追蹤記錄的對話已提交。您可以從「範圍名稱」部分查看，並篩除代理專屬的範圍（名稱為 `agent_run [weather_agent]`）。



如果已篩選跨度，您也可以直接檢查每項追蹤記錄。系統會顯示代理執行的每項動作所花費的詳細時間。舉例來說，請參考下圖



在每個部分中，您都可以檢查屬性中的詳細資料，如下所示



這樣一來，我們就能清楚掌握代理程式與使用者每次互動的資訊，有助於偵錯。歡迎嘗試各種工具或工作流程！

步驟 12 · 約 0 分鐘

12. 🎯 挑戰

嘗試多代理或代理工作流程，瞭解這些流程在負載下的效能，以及追蹤記錄的外觀

步驟 13 · 約 0 分鐘

13. 清理

如要避免系統向您的 Google Cloud 帳戶收取本程式碼研究室所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面。
 2. 在專案清單中選取要刪除的專案，然後點按「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關機) 即可刪除專案。
 4. 或者，您也可以前往控制台的「Cloud Run」，選取剛部署的服務並刪除。
-